

03_mapping_stats

August 31, 2026

```
[1]: import re
import pandas as pd
import os
import glob
import matplotlib.pyplot as plt
import seaborn as sns

from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"
```

```
[23]: def extract_stats(file_path):
    """Extract read statistics from a file."""
    with open(file_path, 'r') as f:
        content = f.read()

    # Extract numbers using regex
    total_umis = int(re.search(r'A total of (\d+) UMIs', content).group(1))
    genes_umis = int(re.search(r'(\d+) \([^\d.]+\%\) correspond to genes',
    ↪content).group(1))
    locus_specific_umis = int(re.search(r'Locus-specific TEs: (\d+) UMIs',
    ↪content).group(1))
    subfamily_umis = int(re.search(r'Subfamily TEs: (\d+)', content).group(1))

    # Get the sample name
    path_parts = file_path.split('/')
    dataset = path_parts[-3] # simulated_mm_RA
    sample = path_parts[-2] # young
    custom_name = f"{dataset}_{sample}"

    return {
        'dataset_sample': custom_name,
        'dataset': dataset,
        'sample': sample,
        'total_umis': total_umis,
        'genes_umis': genes_umis,
        'locus_specific_umis': locus_specific_umis,
        'subfamily_umis': subfamily_umis
```

```

}

def process_files(file_paths):
    """Process a list of file paths."""
    stats_list = []

    for file_path in file_paths:
        try:
            stats = extract_stats(file_path)
            stats_list.append(stats)
        except Exception as e:
            print(f"Error processing {file_path}: {e}")

    # Create DataFrame
    df = pd.DataFrame(stats_list)

    return df

```

```

[24]: SoloTEpath="/mnt/volume_1p5T/results/SoloTEout"
dataset="simulated_mm_RA"
samples=["all", "old", "young"]

file_list = [f"{SoloTEpath}/{dataset}/{sample}/{sample}_SoloTE.stats" for
↪sample in samples]
file_list

[24]: ['/mnt/volume_1p5T/results/SoloTEout/simulated_mm_RA/all/all_SoloTE.stats',
'/mnt/volume_1p5T/results/SoloTEout/simulated_mm_RA/old/old_SoloTE.stats',
'/mnt/volume_1p5T/results/SoloTEout/simulated_mm_RA/young/young_SoloTE.stats']

```

```

[25]: SoloTEpath="/mnt/volume_1p5T/results/SoloTEout"
dataset="simulated_mm_RA"
samples=["all", "old", "young"]

file_list = [f"{SoloTEpath}/{dataset}/{sample}/{sample}_SoloTE.stats" for
↪sample in samples]

df = process_files(file_list)
df
# df.to_csv("output_stats.csv", index=False)

# file_path = "/mnt/volume_1p5T/results/SoloTEout/simulated_mm_RA/young/
↪young_SoloTE.stats"

```

```

[25]:
      dataset_sample      dataset sample  total_umis  genes_umis
0  simulated_mm_RA_all  simulated_mm_RA    all    29868782    340013 \

```

1	simulated_mm_RA_old	simulated_mm_RA	old	32682993	308178
2	simulated_mm_RA_young	simulated_mm_RA	young	26785911	178809

	locus_specific_umis	subfamily_umis
0	19464231	10064538
1	31623787	751028
2	3923579	22683523

```
[26]: df['locus_specific_pct'] = df['locus_specific_umis'] / (
    (df['locus_specific_umis'] + df['subfamily_umis']) * 100
df['subfamily_pct'] = 100 - df['locus_specific_pct']

melted_df = pd.melt(df,
    id_vars=['dataset_sample', 'dataset', 'sample'],
    value_vars=['locus_specific_pct', 'subfamily_pct'],
    var_name='te_type',
    value_name='percentage'
)
melted_df
```

```
[26]:      dataset_sample      dataset sample      te_type
0  simulated_mm_RA_all  simulated_mm_RA    all  locus_specific_pct \
1  simulated_mm_RA_old  simulated_mm_RA    old  locus_specific_pct
2  simulated_mm_RA_young simulated_mm_RA    young locus_specific_pct
3  simulated_mm_RA_all  simulated_mm_RA    all      subfamily_pct
4  simulated_mm_RA_old  simulated_mm_RA    old      subfamily_pct
5  simulated_mm_RA_young simulated_mm_RA    young      subfamily_pct

percentage
0    65.916161
1    97.680209
2    14.746360
3    34.083839
4     2.319791
5    85.253640
```

```
[30]: sns.set_context("poster")
sns.set_style("dark")
# Set up the figure
plt.figure(figsize=(5.3, 7))

df['sample'] = df['sample'].replace({
    'old': 'old TEs',
    'young': 'young TEs',
    'all': 'old+young TEs'
})
```

```

# Create stacked bar chart directly with matplotlib
ax = plt.subplot(111)
bar_width = 0.8

# Plot locus-specific bars
ax.bar(df['sample'], df['locus_specific_pct'], bar_width,
      label='Unique', color='#76ABDF')

# Plot subfamily bars, stacked on top
ax.bar(df['sample'], df['subfamily_pct'], bar_width,
      bottom=df['locus_specific_pct'], label='Multiple', color='#00487C')

# Customizing the plot
plt.title('', fontsize=16)
plt.xlabel('Simulations')
plt.ylabel("% of reads")
plt.xticks(rotation=45, ha='right')
plt.legend(title='Mapping')
plt.tight_layout()

plt.savefig("figures/percMultimapping_" + dataset + "_poster.png")
plt.savefig("figures/percMultimapping_" + dataset + "_poster.pdf")

```

[30]: <Figure size 530x700 with 0 Axes>

[30]: <BarContainer object of 3 artists>

[30]: <BarContainer object of 3 artists>

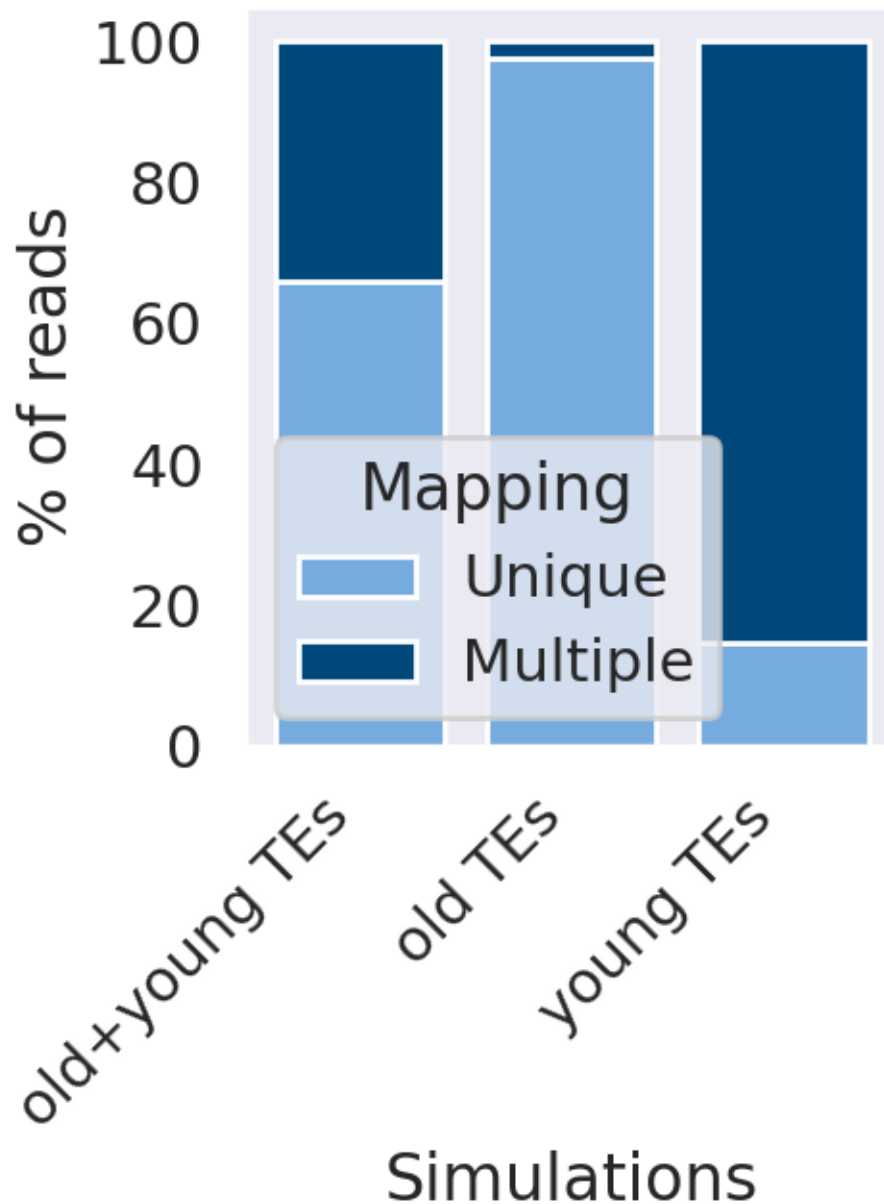
[30]: Text(0.5, 1.0, '')

[30]: Text(0.5, 0, 'Simulations')

[30]: Text(0, 0.5, '% of reads')

[30]: ([0, 1, 2],
 [Text(0, 0, 'old+young TEs'), Text(1, 0, 'old TEs'), Text(2, 0, 'young TEs')])

[30]: <matplotlib.legend.Legend at 0x7f3fa3b25ac0>



```
[29]: sns.set_context("talk")
sns.set_style("dark")
# Set up the figure
plt.figure(figsize=(4, 5.5))

df['sample'] = df['sample'].replace({
    'old': 'old TES',
    'young': 'young TES',
    'all': 'old+young TES'
```

```

})
# order = ['old TEs', 'young TEs', 'old+young TEs']
# df['sample'] = pd.Categorical(df['sample'], categories=order, ordered=True)
# df = df.sort_values('sample')

# Create stacked bar chart directly with matplotlib
ax = plt.subplot(111)
bar_width = 0.8

# Plot locus-specific bars
ax.bar(df['sample'], df['locus_specific_pct'], bar_width,
      label='Unique', color='#76ABDF')

# Plot subfamily bars, stacked on top
ax.bar(df['sample'], df['subfamily_pct'], bar_width,
      bottom=df['locus_specific_pct'], label='Multiple', color='#00487C')

# Customizing the plot
plt.title('', fontsize=16)
plt.xlabel('Simulation')
plt.ylabel("% of reads")
plt.xticks(rotation=45, ha='right')
plt.legend(title='Mapping')
plt.tight_layout()

plt.savefig("figures/percMultimapping_" + dataset + "_talk.png")
plt.savefig("figures/percMultimapping_" + dataset + "_talk|.pdf")

```

[29]: <Figure size 400x550 with 0 Axes>

[29]: <BarContainer object of 3 artists>

[29]: <BarContainer object of 3 artists>

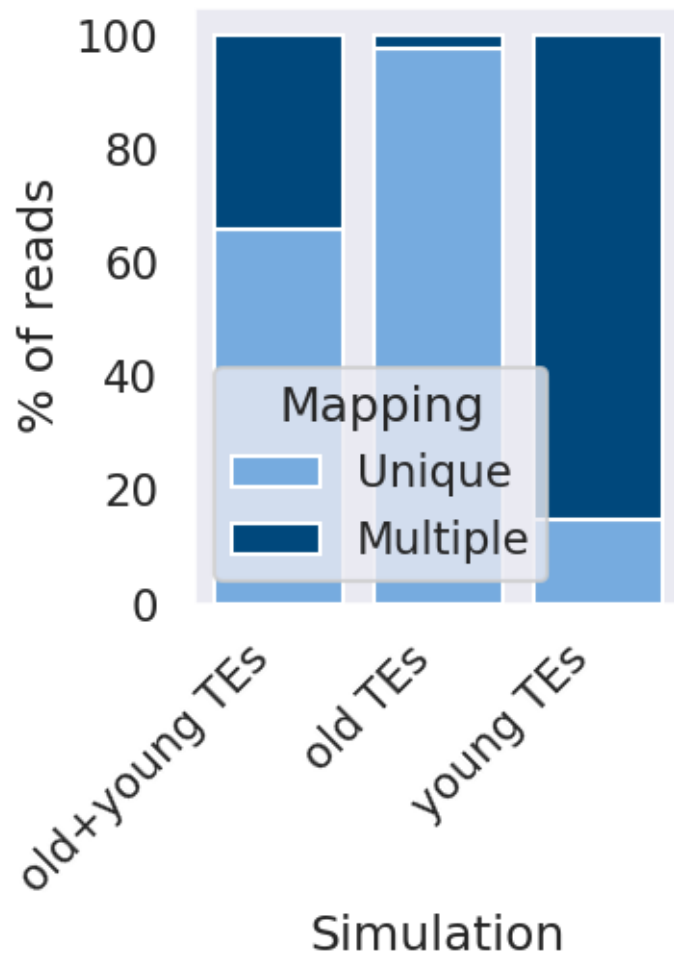
[29]: Text(0.5, 1.0, '')

[29]: Text(0.5, 0, 'Simulation')

[29]: Text(0, 0.5, '% of reads')

[29]: ([0, 1, 2],
 [Text(0, 0, 'old+young TEs'), Text(1, 0, 'old TEs'), Text(2, 0, 'young TEs')])

[29]: <matplotlib.legend.Legend at 0x7f3fa3beb970>



[]:

[]: